Helix OTA

Helix OTA 2.0.0 — Multi-OS Universal Design Document

> **Document ID:** `HELOTA-UNIVERSAL-001` **Version:** 2.0.0-draft **Status:** Active **Last Updated:** 2026-03-05 **Constitution Reference:** HelixConstitution v1 §1–§4, §7.1, §11.4.108 **Prerequisite Versions:** 1.0.0-MVP, 1.0.1-rollback, 1.0.2-delta-updates, 1.1.0-linux-support, 1.2.0-windows-support

---

Table of Contents

---

1. Universal OTA Vision

1.1 The Single-Platform Thesis

The Helix OTA system was architected from its inception with a singular long-term thesis: **there should be exactly one OTA platform for every operating system in a fleet.** Versions 1.0.0 through 1.2.0 validated this thesis incrementally — first with Android (1.0.0), then Linux (1.1.0), then Windows (1.2.0). Each of those versions introduced an OS adapter that implemented a common interface, but the adapters were statically compiled into the server binary. Version 2.0.0 completes the vision by extracting the adapter interface into a **dynamically loaded plugin system**, making it possible to add support for any operating system without recompiling or redeploying the core server.

The universal OTA platform eliminates three classes of operational pain that plague organizations managing heterogeneous device fleets:

1. **Toolchain fragmentation.** Organizations today run separate OTA systems for Android (Google Omaha, private A/B update servers), Linux (Mender, OSTree, custom apt repositories), Windows (WSUS, SCCM), and embedded devices (vendor-specific bootloaders). Each system has its own dashboard, its own rollout logic, its own artifact format, and its own authentication model. Helix OTA 2.0.0 replaces all of these with a single control plane.

2. **Cross-OS coordination failure.** When an Android app update requires a corresponding backend API version, and that backend runs on Linux servers, and the API gateway runs on Windows — there is no unified mechanism to coordinate the rollout across all three OS types. Helix OTA 2.0.0 introduces cross-OS dependency management and coordinated rollout, ensuring that related updates across different OS targets are applied in the correct order.

3. **Duplicate infrastructure.** Each separate OTA system requires its own artifact storage, device registry, telemetry pipeline, authentication layer, and dashboard. Helix OTA 2.0.0 shares all of this infrastructure across every OS adapter, reducing operational overhead by an estimated 60–80% compared to running three or more separate OTA systems.

1.2 Core Design Principles

The 2.0.0 architecture extends the principles established in 1.0.0 with two new principles specific to the universal platform:

#	Principle	Definition	Manifestation in 2.0.0
P1	**Anti-Bluff**	No feature declared complete without mutation-tested evidence	Every adapter plugin must pass the adapter conformance test suite with ≥ 85% mutation score before registration
P2	**Test-First**	Tests before implementation	Plugin SDK conformance tests define the contract; adapters are validated against them
P3	**Nano-Detail Docs**	Every API, model, and transition documented to specification grade	Plugin manifest schema, adapter lifecycle state machine, and cross-OS dependency graph are all formally specified
P4	**Fail-Safe by Default**	Updates never destroy the running system	Every adapter must implement atomic install with rollback; no best-effort installs
P5	**Minimal Trusted Compute Base**	Only the update engine and our client touch partitions	Plugin sandboxing restricts adapter access to only the resources declared in the manifest
P6	**Submodule Reuse**	Never re-implement solved problems	All adapters share the same vasic-digital infrastructure layer
P7	**Progressive Delivery**	Rollout 0→100% in configurable increments	Cross-OS rollout coordination respects per-OS rollout speed
P8	**Plugin Isolation**	A failing adapter must never compromise	Plugins run in sandboxed processes; communication via

#	Principle	Definition	Manifestation in 2.0.0
		the server or other adapters	typed RPC; resource limits enforced
P9	**Zero-Down Adapter Addition**	Adding a new OS adapter must not require server restart	Plugins are loaded at runtime via the adapter registry; hot-load and hot-swap are supported

1.3 Plugin Architecture: The Abstraction That Makes It Universal

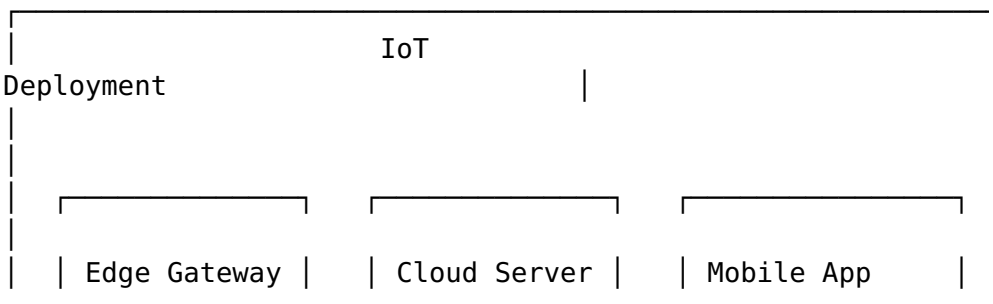
The key architectural insight of 2.0.0 is that **every OS-specific update mechanism can be reduced to a finite set of operations on a finite set of artifact types.** An Android A/B update via `update_engine`, a Linux `rpm-ostree` upgrade, a Windows MSI installation, and a FreeRTOS firmware flash are all fundamentally the same operation: take an artifact, verify its integrity and authenticity, install it atomically, and report the result. The differences lie in:

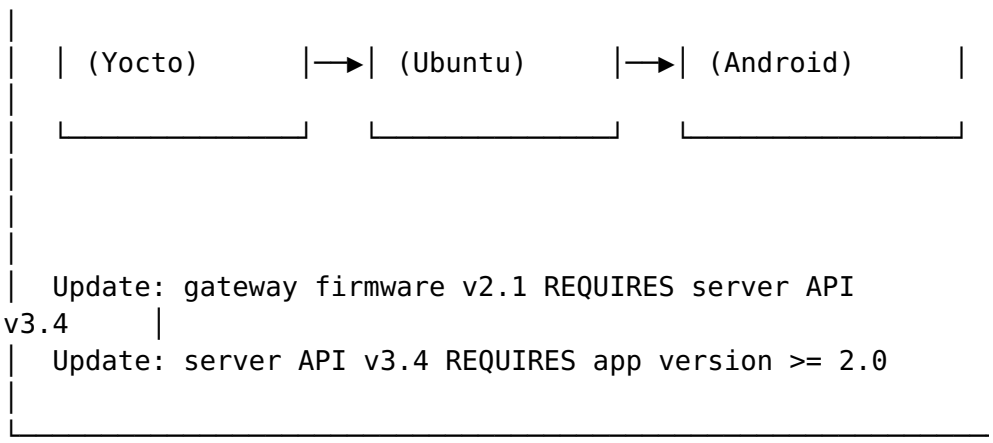
- The artifact format (ZIP, RPM, MSI, firmware binary, OSTree commit)
- The verification method (SHA-256 + RSA, code-signing certificate, OSTree GPG signature)
- The install mechanism (`update_engine` Binder IPC, `rpm-ostree upgrade`, `msiexec`, bootloader flash)
- The rollback mechanism (A/B slot swap, `rpm-ostree rollback`, MSI uninstall, dual-bank switch)

The plugin architecture encapsulates these differences behind the `OSAdapter` interface. The server's update pipeline becomes OS-agnostic: it invokes the same lifecycle stages regardless of the target OS, delegating the OS-specific implementation to the loaded adapter plugin.

1.4 Cross-OS Dependency Management

A fleet rarely consists of a single OS type. Consider a typical IoT deployment:





Without cross-OS dependency management, the gateway firmware updates first and breaks because the server API is still on v3.3. With cross-OS dependency management, the Helix OTA server constructs a dependency graph across OS types and enforces update ordering: server first, then gateway, then app.

The dependency graph is declared in the rollout configuration:

```
rollout:
  name: "iot-stack-v2.1"
  stages:
    - name: "server-upgrade"
      os_type: "ubuntu_server"
      artifact_id: "art_server_v3.4"
      target_percentage: 100
      gate: "success_rate >= 99%"
    - name: "gateway-upgrade"
      os_type: "yocto"
      artifact_id: "art_gateway_v2.1"
      depends_on: ["server-upgrade"]
      target_percentage: 50
    - name: "app-upgrade"
      os_type: "android"
      artifact_id: "art_app_v2.0"
      depends_on: ["gateway-upgrade"]
      target_percentage: 100
```

Each stage has an explicit `depends_on` list. A stage does not begin until all its dependencies have reached their target percentage with an acceptable success rate. This is the **cross-OS rollout coordination** mechanism.

---

2. Plugin Architecture Design

2.1 Plugin Runtime: Dual-Mode Architecture

Go's native `plugin` package (`plugin.Open`) has well-documented limitations: it requires the plugin and the host to be compiled with the exact same Go version and build flags, it only works on Linux and macOS (not Windows), and it provides no isolation between the plugin and the host process. These limitations make it unsuitable as the sole plugin runtime for a universal platform.

Helix OTA 2.0.0 implements a **dual-mode plugin architecture**:

Mode	Runtime	Isolation	Performance	Platform Support	Use Case
Native	Go `plugin.Open`	Process-level (shared memory)	≤ 0.1ms per call	Linux, macOS	First-party adapters where performance is critical
WASM	wazero (pure Go WASM runtime)	Sandbox (capability-based)	≤ 1ms per call	All platforms	Third-party adapters where isolation is critical

The server detects the plugin type from the manifest and loads it using the appropriate runtime. First-party adapters (Android, Linux, Windows) ship in both native and WASM formats. Third-party adapters are recommended to use WASM for portability and isolation.

2.2 OSAdapter Interface — Full Definition

The `OSAdapter` interface is the central contract of the plugin system. Every adapter must implement it:

```
package pluginsdk

import (
    "context"
    "io"
    "time"
)

// OSAdapter defines the contract for OS-specific update
        logic.
```

```go
// Every adapter plugin must implement this interface.
// The interface is versioned; adapters declare their
    implemented version
// in the plugin manifest.
type OSAdapter interface {
    // Metadata returns the adapter's identity and
        capabilities.
    // Called once during adapter registration. Must not
        block.
    Metadata() AdapterMetadata

    // Initialize configures the adapter with OS-specific
        settings.
    // Called once after registration, before any other
        method.
    // The config map contains adapter-specific key-value
        pairs
    // from the server configuration file.
    Initialize(ctx context.Context, config
        map[string]interface{}) error

    // CheckForUpdate queries the device's current state and
        determines
    // whether an update is available for this OS type.
    // The device parameter contains all known device
        metadata.
    // Returns nil if no update is available.
    CheckForUpdate(ctx context.Context, device DeviceInfo)
        (*UpdateInfo, error)

    // PrepareArtifact validates that the artifact is
        compatible with
    // the target device. This is a server-side operation —
        the adapter
    // verifies the artifact format and compatibility
        without the device.
    PrepareArtifact(ctx context.Context, artifact
        ArtifactInfo, device DeviceInfo)
        (*PreparedArtifact, error)

    // ApplyUpdate triggers the OS-specific update mechanism
        on the device.
    // The artifact has already been downloaded and verified
        by the
    // universal pipeline. The adapter is responsible for
        invoking the
    // OS-specific install mechanism.
    ApplyUpdate(ctx context.Context, device DeviceInfo,
        artifact PreparedArtifact, progressChan chan<-
        Progress) error

    // VerifyUpdate confirms the update was applied
        correctly after reboot.
    // Called after the device reports REBOOT_COMPLETE.
```

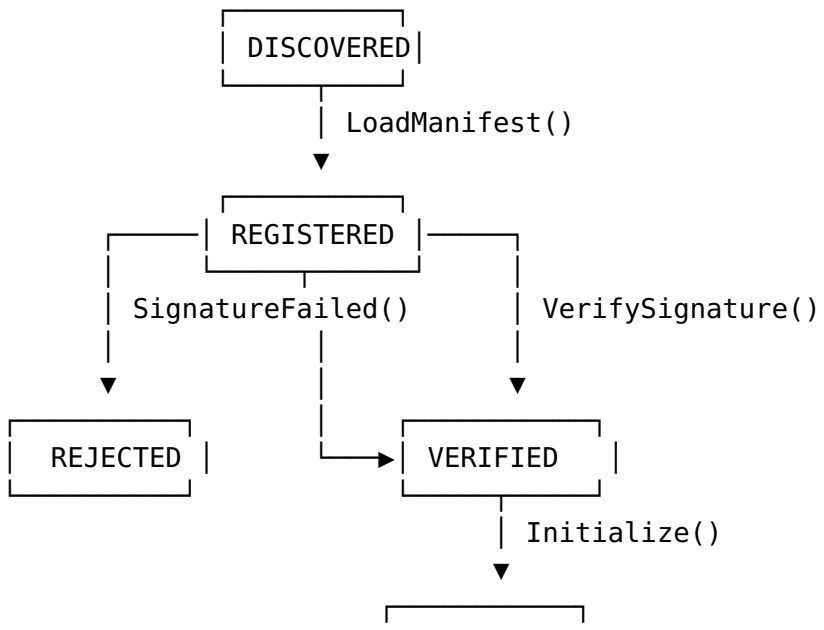
```
    VerifyUpdate(ctx context.Context, device DeviceInfo)
        (*VerificationResult, error)

    // Rollback reverts to the previous system state.
    // Must be safe to call at any point during the update
        lifecycle.
    // If rollback is not supported, return
        ErrRollbackNotSupported.
    Rollback(ctx context.Context, device DeviceInfo) error

    // HealthCheck verifies the adapter is operational.
    // Called periodically by the server's health monitor.
    HealthCheck(ctx context.Context) error

    // Shutdown performs cleanup before the adapter is
        unloaded.
    // Called during graceful server shutdown or adapter
        hot-swap.
    Shutdown(ctx context.Context) error
}

// AdapterMetadata describes an adapter's identity and
        capabilities.
type AdapterMetadata struct {
    Name         string   `json:"name"`          // e.g.,
        "helix-adapter-freebsd"
    Version      string
        `json:"version"`       // Semantic version, e.g.,
        "1.0.0"
    OSType       string   `json:"os_type"`       // e.g.,
        "freebsd", "android", "linux_debian"
    OSDisplayName string  `json:"os_display_name"` // e.g.,
        "FreeBSD", "Android 15"
    Capabilities []string `json:"capabilities"`  // e.g.,
        ["full_update", "delta_update", "rollback"]
    APIVersion   string   `json:"api_version"`   // Must
        match server's supported plugin API version
    Author       string   `json:"author"`
    License      string   `json:"license"`
    Homepage     string   `json:"homepage"`
    MinServerVersion string `json:"min_server_version"` //
        Minimum Helix OTA server version
}

// DeviceInfo contains device metadata passed to adapter
        methods.
type DeviceInfo struct {
    DeviceID           string
        `json:"device_id"`
    OSType             string
        `json:"os_type"`
    OSVersion          string
        `json:"os_version"`
    HardwareModel      string
        `json:"hardware_model"`
```

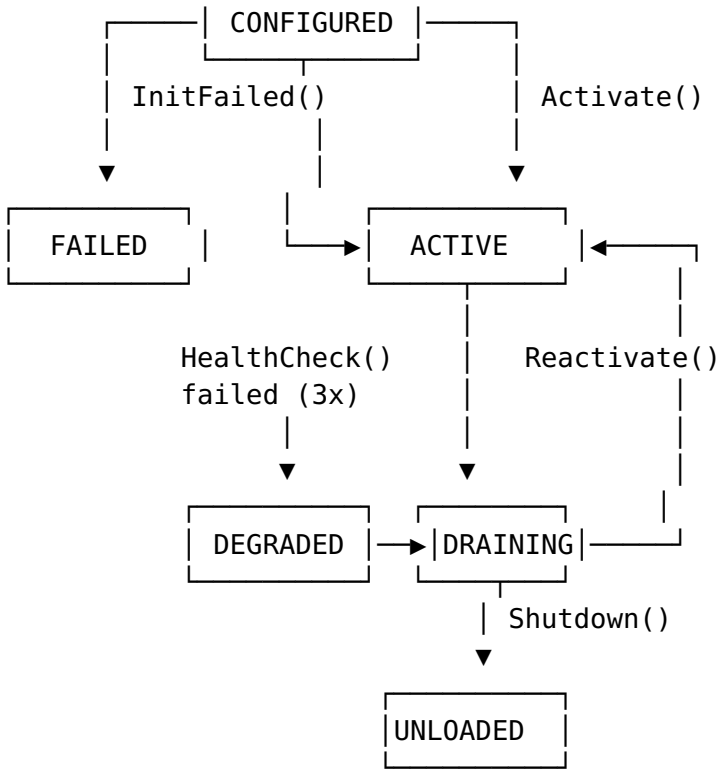
```
    CurrentVersion     string
        `json:"current_version"`
    HardwareFingerprint string
        `json:"hardware_fingerprint"`
    Metadata           map[string]interface{}
        `json:"metadata"` // OS-specific fields
    LastCheckIn        time.Time
        `json:"last_check_in"`
}

// UpdateInfo describes an available update for a device.
type UpdateInfo struct {
    Available       bool   `json:"available"`
    ArtifactID      string `json:"artifact_id"`
    TargetVersion   string `json:"target_version"`
    UpdateType      string `json:"update_type"` // "full",
        "delta", "patch"
    SizeBytes       int64  `json:"size_bytes"`
    ChangelogURL    string `json:"changelog_url,omitempty"`
    Mandatory       bool   `json:"mandatory"`
    Deadline        *time.Time `json:"deadline,omitempty"`
}

// ArtifactInfo describes an uploaded artifact before
        preparation.
type ArtifactInfo struct {
    ArtifactID      string   `json:"artifact_id"`
    Filename        string   `json:"filename"`
    TargetVersion   string   `json:"target_version"`
    MinSourceVersion string  `json:"min_source_version"`
    SHA256          string   `json:"sha256"`
    SizeBytes       int64    `json:"size_bytes"`
    ArtifactType    string   `json:"artifact_type"` //
        "zip", "msi", "rpm", "deb", "firmware",
        "ostree_commit"
    TargetModels    []string `json:"target_models"`
}

// PreparedArtifact is an artifact that has been validated
        by the adapter
// for a specific device. It may contain adapter-specific
        metadata
// needed for installation.
type PreparedArtifact struct {
    ArtifactID      string
        `json:"artifact_id"`
    DownloadURL     string
        `json:"download_url"`
    SHA256          string                  `json:"sha256"`
    SizeBytes       int64
        `json:"size_bytes"`
    InstallParams   map[string]interface{}
        `json:"install_params"` // Adapter-specific
}
```

```go
// VerificationResult confirms whether an update was applied
        correctly.
type VerificationResult struct {
    Success      bool   `json:"success"`
    NewVersion   string `json:"new_version"`
    ErrorMessage string `json:"error_message,omitempty"`
}

// Progress reports update application progress.
type Progress struct {
    Stage           string
        `json:"stage"` // "downloading", "verifying",
        "installing", "finalizing"
    ProgressPercent float64 `json:"progress_percent"`
    Message         string  `json:"message,omitempty"`
}

// Sentinel errors
var (
    ErrRollbackNotSupported = errors.New("rollback not
        supported by this adapter")
    ErrAdapterNotReady      = errors.New("adapter not
        initialized")
    ErrArtifactIncompatible = errors.New("artifact
        incompatible with device")
    ErrUpdateInProgress     = errors.New("update already in
        progress for this device")
)
```

2.3 Adapter Lifecycle State Machine

Every adapter follows the same lifecycle, managed by the server's `AdapterManager`:





State Descriptions:

State	Description	Transitions
`DISCOVERED`	Manifest found on disk or in registry	→ `REGISTERED` (manifest valid), → `REJECTED` (manifest invalid)
`REGISTERED`	Manifest parsed and basic validation passed	→ `VERIFIED` (signature valid), → `REJECTED` (signature invalid)
`VERIFIED`	Plugin signature verified against trusted CA	→ `CONFIGURED` (initialization succeeds), → `FAILED` (initialization fails)
`CONFIGURED`	Adapter initialized with server configuration	→ `ACTIVE` (activation succeeds), → `FAILED` (activation fails)
`ACTIVE`	Adapter is processing update requests	→ `DEGRADED` (3 consecutive health check failures), → `DRAINING` (shutdown initiated)
`DEGRADED`	Adapter is unhealthy; new requests routed to fallback	→ `ACTIVE` (health check recovers), → `DRAINING` (manual intervention)
`DRAINING`		

State	Description	Transitions
	Adapter is finishing in-flight requests; no new requests	→ `UNLOADED` (all in-flight requests complete or timeout)
`UNLOADED`	Adapter is fully stopped and resources released	→ `DISCOVERED` (re-registration after update)
`REJECTED`	Adapter failed validation; cannot be used	Terminal state (requires manual intervention)
`FAILED`	Adapter failed initialization or activation	→ `CONFIGURED` (retry after configuration fix)

2.4 Adapter Manager Implementation

```
package adapter

import (
    "context"
    "fmt"
    "sync"
    "time"

    "dev.helix.ota.server/pluginsdk"
)

// AdapterManager manages the lifecycle of all loaded OS
        adapters.
type AdapterManager struct {
    mu         sync.RWMutex
    adapters   map[string]*managedAdapter // key: os_type
    registry   AdapterRegistry
    healthTick time.Duration
    logger     *zap.Logger
}

type managedAdapter struct {
    adapter    pluginsdk.OSAdapter
    state      AdapterState
    loadedAt   time.Time
    failCount  int
    metrics    AdapterMetrics
}

// AdapterMetrics tracks runtime metrics for an adapter.
type AdapterMetrics struct {
    TotalRequests     int64
    SuccessfulUpdates int64
```

```
    FailedUpdates     int64
    AverageLatency    time.Duration
    LastHealthCheck   time.Time
}

// LoadAndRegister discovers, validates, and activates an
        adapter plugin.
func (m *AdapterManager) LoadAndRegister(ctx
        context.Context, pluginPath string) error {
    // 1. Load and parse manifest
    manifest, err := ParseManifest(pluginPath + "/
        plugin.yaml")
    if err != nil {
        return fmt.Errorf("parse manifest: %w", err)
    }

    // 2. Verify plugin signature
    if err := VerifyPluginSignature(pluginPath, manifest);
        err != nil {
        return fmt.Errorf("signature verification failed:
        %w", err)
    }

    // 3. Load plugin (native or WASM based on manifest)
    var adapter pluginsdk.OSAdapter
    if manifest.Runtime == "native" {
        adapter, err = LoadNativePlugin(pluginPath)
    } else {
        adapter, err = LoadWASMPlugin(pluginPath)
    }
    if err != nil {
        return fmt.Errorf("load plugin: %w", err)
    }

    // 4. Validate API version compatibility
    metadata := adapter.Metadata()
    if !isAPIVersionCompatible(metadata.APIVersion,
        CurrentPluginAPIVersion) {
        return fmt.Errorf("incompatible API version:
        adapter=%s, server=%s",
            metadata.APIVersion, CurrentPluginAPIVersion)
    }

    // 5. Initialize adapter with server configuration
    config, err := m.getAdapterConfig(manifest.OSType)
    if err != nil {
        return fmt.Errorf("load adapter config: %w", err)
    }
    if err := adapter.Initialize(ctx, config); err != nil {
        return fmt.Errorf("adapter initialization failed:
        %w", err)
    }
```

```
    // 6. Health check before activation
    if err := adapter.HealthCheck(ctx); err != nil {
        return fmt.Errorf("adapter health check failed:
        %w", err)
    }

    // 7. Register as active
    m.mu.Lock()
    osType := metadata.OSType

    // Gracefully drain existing adapter if hot-swapping
    if existing, ok := m.adapters[osType]; ok {
        m.mu.Unlock()
        m.drainAdapter(ctx, existing)
        m.mu.Lock()
    }

    m.adapters[osType] = &managedAdapter{
        adapter:  adapter,
        state:    StateActive,
        loadedAt: time.Now(),
    }
    m.mu.Unlock()

    m.logger.Info("adapter registered and activated",
        "os_type", osType,
        "name", metadata.Name,
        "version", metadata.Version,
    )

    return nil
}

// GetAdapter returns the active adapter for the given OS
        type.
// Returns an error if no adapter is registered or the
        adapter is not active.
func (m *AdapterManager) GetAdapter(osType string)
        (pluginsdk.OSAdapter, error) {
    m.mu.RLock()
    defer m.mu.RUnlock()

    managed, ok := m.adapters[osType]
    if !ok {
        return nil,
        fmt.Errorf("no adapter registered for OS type:
        %s", osType)
    }
    if managed.state != StateActive && managed.state !=
        StateDegraded {
        return nil, fmt.Errorf("adapter for %s is in state
        %s, not available",
            osType, managed.state)
```

```
    }
    return managed.adapter, nil
}

// drainAdapter gracefully shuts down an adapter, waiting
       for in-flight
// requests to complete or timing out after 30 seconds.
func (m *AdapterManager) drainAdapter(ctx context.Context,
       ma *managedAdapter) {
    drainCtx, cancel := context.WithTimeout(ctx,
       30*time.Second)
    defer cancel()
    ma.adapter.Shutdown(drainCtx)
    ma.state = StateUnloaded
}
```

2.5 Adapter Marketplace / Registry

The `helix-plugin-registry` is both a local filesystem-based registry (for air-gapped deployments) and a cloud-based registry (for connected deployments). The registry stores:

1. **Plugin binaries** — the compiled `.so` (native) or `.wasm` (WASM) files
2. **Plugin manifests** — the `plugin.yaml` metadata files
3. **Plugin signatures** — RSA-4096 signatures over the binary + manifest
4. **Plugin versions** — semantic version history with changelogs

```
// AdapterRegistry defines the interface for plugin storage
       and distribution.
type AdapterRegistry interface {
    // ListAvailable returns all adapters available in the
       registry.
    ListAvailable(ctx context.Context) ([]RegistryEntry,
       error)

    // Download downloads an adapter plugin to the local
       plugin directory.
    Download(ctx context.Context, name string, version
       string) (string, error)

    // Upload publishes an adapter plugin to the registry.
    Upload(ctx context.Context, entry RegistryEntry,
       binaryPath string, manifestPath string) error

    // GetVersions returns all available versions of an
       adapter.
    GetVersions(ctx context.Context, name string)
       ([]string, error)
```

```
    // VerifyIntegrity checks the integrity of a downloaded
        plugin.
    VerifyIntegrity(ctx context.Context, name string,
        version string) error
}

type RegistryEntry struct {
    Name         string    `json:"name"`
    Version      string    `json:"version"`
    OSType       string    `json:"os_type"`
    Author       string    `json:"author"`
    Signature    string    `json:"signature"`
    PublishedAt  time.Time `json:"published_at"`
    DownloadURL  string    `json:"download_url"`
    ChecksumSHA256 string  `json:"checksum_sha256"`
    SizeBytes    int64     `json:"size_bytes"`
    ChangelogURL string    `json:"changelog_url"`
}
```

2.6 Third-Party Adapter Development SDK

The `helix-plugin-sdk` provides everything a third-party developer needs to build an adapter:

1. **Go module** — `dev.helix.ota.pluginsdk` containing the `OSAdapter` interface, all type definitions, and helper functions
2. **Conformance test suite** — `pluginsdk.ConformanceSuite` that validates an adapter against the full contract
3. **Adapter scaffolding tool** — `helix-adapter-init` CLI that generates a new adapter project with boilerplate
4. **WASM compilation toolchain** — `helix-adapter-build` CLI that compiles a Go adapter to WASM with the correct flags
5. **Local testing harness** — `helix-adapter-test` CLI that runs an adapter against a mock Helix OTA server

```
// ConformanceSuite validates that an adapter implements the
        OSAdapter
// interface correctly. Third-party adapters must pass all
        conformance
// tests before they can be registered in the plugin
        registry.
type ConformanceSuite struct {
    Adapter OSAdapter
    T       *testing.T
}

// RunAll executes the full conformance test suite.
func (cs *ConformanceSuite) RunAll() {
    cs.TestMetadataReturnsValidStruct()
    cs.TestInitializeAcceptsConfig()
```

```
    cs.TestInitializeRejectsInvalidConfig()
    cs.TestCheckForUpdateReturnsUpdateInfo()
    cs.TestCheckForUpdateReturnsNilWhenNoUpdate()
    cs.TestPrepareArtifactValidatesCompatibility()
    cs.TestPrepareArtifactRejectsIncompatibleArtifact()
    cs.TestApplyUpdateReportsProgress()
    cs.TestApplyUpdateReturnsErrorOnFailure()
    cs.TestVerifyUpdateConfirmsSuccess()
    cs.TestVerifyUpdateDetectsFailure()
    cs.TestRollbackRevertsToPreviousState()
    cs.TestRollbackReturnsErrNotSupportedIfUnsupported()
    cs.TestHealthCheckReturnsNilWhenHealthy()
    cs.TestHealthCheckReturnsErrorWhenUnhealthy()
    cs.TestShutdownCleansUpResources()
    cs.TestConcurrentCallsAreSafe()
    cs.TestContextCancellationIsRespected()
}
```

---

3. Supported Operating Systems Catalog

3.1 Mobile Operating Systems

OS	Adapter	Update Mechanism	Artifact Format	Delta Support	Rollback	Sta
Android (AOSP)	`helix-adapter-android` (1.0.0)	`update_engine` A/B partition	OTA ZIP (payload.bin)	Yes (1.0.2)	A/B slot swap	**Sta**
Android (GMS)	`helix-adapter-android`	Same as AOSP + Google Play integrity	OTA ZIP	Yes	A/B slot swap	**Sta**
Android (custom ROM)	`helix-adapter-android`	`update_engine` or recovery-based	OTA ZIP	Partial	Varies by ROM	**Be eff**
iOS	Research only	Apple MDM + OTA	IPA (via App Store)	No	Apple-controlled	**Re** — res OS OT Ap ow ser He onl ma

OS	Adapter	Update Mechanism	Artifact Format	Delta Support	Rollback	Sta
						app upd via

iOS Research Note: Apple's ecosystem is fundamentally closed to third-party OS-level OTA. The iOS adapter (planned for 2.3.0) will operate within Apple's constraints: it will manage app-level updates via MDM push commands, supervise OS update policies (defer, enforce deadline), and report compliance. It cannot replace Apple's OTA mechanism for iOS itself.

3.2 Desktop Operating Systems

OS	Adapter	Update Mechanism	Artifact Format	Delta Support	Rollback	Stat
Windows 10	`helix-adapter-windows` (1.2.0)	Windows Update API + MSI/MSIX	MSI, MSIX, CAB	No	MSI uninstall, System Restore	**Stal**
Windows 11	`helix-adapter-windows`	Same as Windows 10	MSI, MSIX	No	Same	**Stal**
macOS	`helix-adapter-macos` (2.1.0)	`softwareupdate`, installer packages	PKG, DMG	No	Time Machine snapshot, APFS snapshot	**Plan**
ChromeOS	`helix-adapter-chromeos`	ChromeOS update engine	Omaha-format payload	Yes	A/B partition	**Futu**
Linux Desktop (Ubuntu)	`helix-adapter-linux` (1.1.0)	APT + dpkg	DEB	Partial (1.0.2)	`apt-get downgrade`	**Stal**
Linux Desktop (Fedora)	`helix-adapter-linux`	rpm-ostree	RPM, OSTree commit	Yes	`rpm-ostree rollback`	**Stal**

3.3 Server Operating Systems

OS	Adapter	Update Mechanism	Artifact Format	Delta Support	Rollback	Status
			DEB	Partial	`apt-get downgrade`,	**Stable**

OS	Adapter	Update Mechanism	Artifact Format	Delta Support	Rollback	Status
Ubuntu Server	`helix-adapter-linux`	APT + unattended-upgrades			A/B rootfs	
RHEL/ CentOS	`helix-adapter-linux`	rpm-ostree + dnf	RPM, OSTree commit	Yes	`rpm-ostree rollback`	**Stable**
Windows Server	`helix-adapter-windows`	WSUS + MSI	MSI, MSIX	No	MSI uninstall	**Stable**
Debian	`helix-adapter-linux`	APT + dpkg	DEB	Partial	`apt-get downgrade`	**Stable**

3.4 Embedded Operating Systems

OS	Adapter	Update Mechanism	Artifact Format	Delta Support	Rollback	Sta
Yocto/ Buildroot	`helix-adapter-yocto` (2.0.0)	SWUpdate, Mender client, or raw flash	SWU, rootfs image, firmware binary	Yes (bsdiff)	Dual-bank (A/B), recovery partition	**Nev**
FreeRTOS	`helix-adapter-rtos` (2.0.0)	MCUboot, custom bootloader	Firmware binary (.bin, .hex)	Yes (bsdiff)	Dual-bank, bootloader fallback	**Nev**
Zephyr	`helix-adapter-rtos`	MCUboot + Zephyr DFU	Firmware binary	Yes	MCUboot revert	**Nev**

3.5 IoT Operating Systems

OS	Adapter	Update Mechanism	Artifact Format	Delta Support	Rollback	Status
Android Things	`helix-adapter-android`	Same as Android A/B	OTA ZIP	Yes	A/B slot swap	**Stable**
Ubuntu Core	`helix-adapter-linux`	snapd + snap refresh	Snap	Yes (delta snap)	`snap revert`	**Stable**
Mbed OS	`helix-adapter-rtos`	Mbed Cloud Update	Firmware binary	Partial	Dual-bank	**Planne**

3.6 Other Operating Systems

OS	Adapter	Update Mechanism	Artifact Format	Delta Support	Rollback
FreeBSD	`helix-adapter-freebsd` (2.0.0)	`freebsd-update`, pkg	TXZ, PKG	No	`freebsd-update rollback`, boot environment (BE)
OpenBSD	`helix-adapter-freebsd`	syspatch, pkg_add	TGZ, PKG	No	Manual reinstall from snapshot
HarmonyOS	`helix-adapter-harmonyos`	HarmonyOS Update Engine	APP (HarmonyOS package)	Planned	A/B partition
Tizen	`helix-adapter-tizen`	Tizen Package Manager	RPM (Tizen variant)	Partial	`pkgcmd rollback`

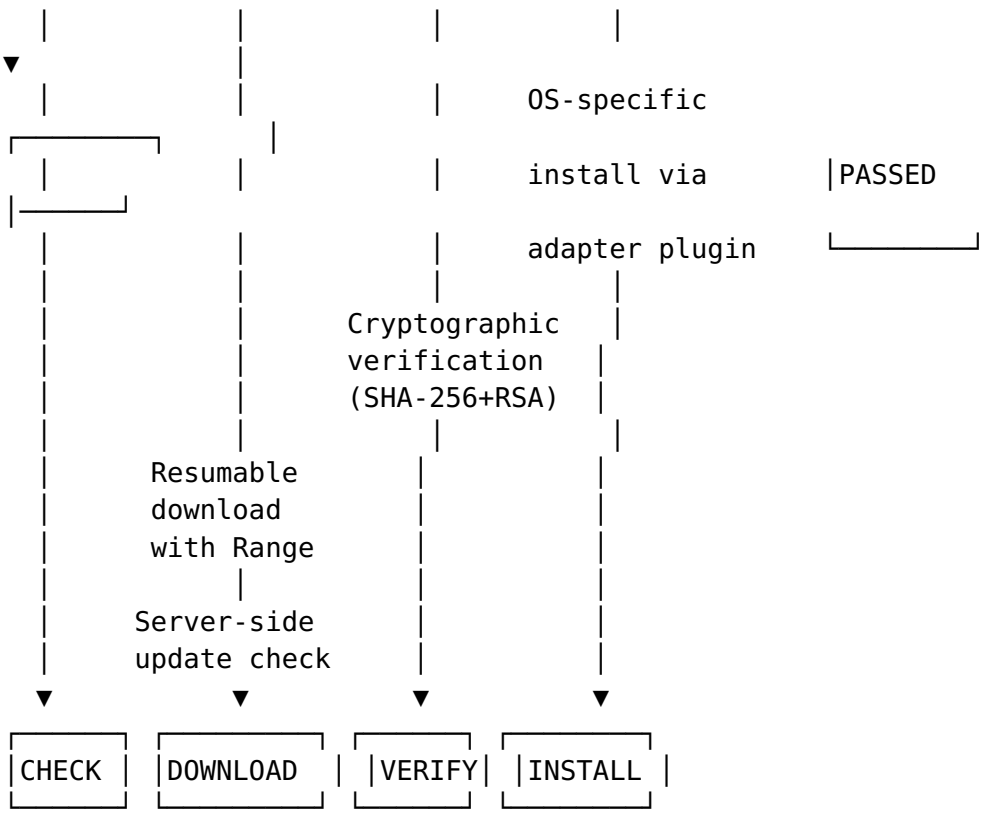
---

4. OS-Agnostic Update Pipeline

4.1 Universal Update Lifecycle

Every update, regardless of OS type, follows the same six-stage lifecycle:

```
CHECK ──► DOWNLOAD ──► VERIFY ──► INSTALL ──► VERIFY_POST
──► COMMIT
  │           │           │          │
│             │
  │           │           │          │
│             ▼
  │           │           │          │
│         [COMPLETE]
  │           │           │          │              │
  │           │           │          │         Post-
install    │
  │           │           │          │
verification    │ ROLLBACK
  │           │           │          │         (health
check,  │ (if VERIFY_POST
  │           │           │          │          boot
verify)    │  fails)
  │           │           │          │
│             │
```



Each stage is implemented as a pipeline step that delegates to the active adapter:

```
package pipeline

// UniversalUpdatePipeline executes the OS-agnostic update
        lifecycle.
type UniversalUpdatePipeline struct {
    adapterMgr *adapter.AdapterManager
    artifact   ArtifactService
    telemetry  TelemetryService
    logger     *zap.Logger
}

// PipelineStage represents a stage in the update lifecycle.
type PipelineStage string

const (
    StageCheck       PipelineStage = "CHECK"
    StageDownload    PipelineStage = "DOWNLOAD"
    StageVerify      PipelineStage = "VERIFY"
    StageInstall     PipelineStage = "INSTALL"
    StageVerifyPost  PipelineStage = "VERIFY_POST"
    StageCommit      PipelineStage = "COMMIT"
)

// Execute runs the full update pipeline for a device.
func (p *UniversalUpdatePipeline) Execute(
    ctx context.Context,
```

```
    device DeviceInfo,
) error {
    // Stage 1: CHECK — Adapter determines if an update is
        available
    adapter, err := p.adapterMgr.GetAdapter(device.OSType)
    if err != nil {
        return fmt.Errorf("get adapter: %w", err)
    }

    updateInfo, err := adapter.CheckForUpdate(ctx, device)
    if err != nil {
        return p.failStage(ctx, device, StageCheck, err)
    }
    if updateInfo == nil || !updateInfo.Available {
        return nil // no update available
    }

    // Stage 2: DOWNLOAD — Universal download (resumable,
        bandwidth-throttled)
    artifact, err := p.artifact.PrepareForDevice(ctx,
        updateInfo.ArtifactID, device)
    if err != nil {
        return p.failStage(ctx, device, StageDownload, err)
    }

    // Stage 3: VERIFY — Cryptographic verification
        (universal)
    if err := p.verifyArtifact(ctx, artifact); err != nil {
        return p.failStage(ctx, device, StageVerify, err)
    }

    // Stage 4: INSTALL — Adapter-specific install mechanism
    progressChan := make(chan pluginsdk.Progress, 10)
    go p.reportProgress(ctx, device, progressChan)

    if err := adapter.ApplyUpdate(ctx, device, artifact,
        progressChan); err != nil {
        // Attempt automatic rollback
        _ = adapter.Rollback(ctx, device)
        return p.failStage(ctx, device, StageInstall, err)
    }

    // Stage 5: VERIFY_POST — Post-install verification
        (adapter-specific)
    result, err := adapter.VerifyUpdate(ctx, device)
    if err != nil || !result.Success {
        _ = adapter.Rollback(ctx, device)
        return p.failStage(ctx, device, StageVerifyPost,
        fmt.Errorf("post-install verification failed: %s",
        result.ErrorMessage))
    }

    // Stage 6: COMMIT — Mark update as successful
```

```
    return p.commitUpdate(ctx, device, result.NewVersion)
}
```

4.2 Adapter-to-Pipeline Mapping

Each OS adapter maps its native update mechanism to the universal pipeline stages:

OS Type	CHECK	DOWNLOAD	VERIFY	INSTALL
Android	`update_engine` status check	HTTP Range download	SHA-256 + RSA on payload.bin	`update_engine.ApplyP`
Linux (APT)	`apt list --upgradable`	HTTP download	SHA-256 + GPG on .deb	`apt-get install`
Linux (rpm-ostree)	`rpm-ostree upgrade --check`	OSTree pull	GPG on OSTree commit	`rpm-ostree upgrade`
Windows	Windows Update Agent scan	BITS download	Authenticode on MSI/ MSIX	`msiexec /i`
FreeBSD	`freebsd-update fetch`	HTTP download	SHA-256 + RSA on TXZ	`freebsd-update insta`
Yocto	SWUpdate check	HTTP download	SHA-256 + RSA on SWU	`swupdate` via IPC
FreeRTOS	Version compare	HTTP download	SHA-256 on firmware bin	MCUboot flash writ

4.3 Server-Side Version Compatibility Matrix

The server maintains a version compatibility matrix that maps source version ranges to target versions for each OS type. This matrix is used during the CHECK stage to determine whether a device is eligible for an update:

```
CREATE TABLE version_compatibility (
    id              UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
    os_type         VARCHAR(64) NOT NULL,
    source_version  VARCHAR(128) NOT NULL,
        -- semver range, e.g., ">=1.0.0 <2.0.0"
    target_version  VARCHAR(128) NOT NULL,
    artifact_id     UUID NOT NULL REFERENCES artifacts(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE(os_type, source_version, target_version)
);
```

```
CREATE INDEX idx_version_compat_os ON
        version_compatibility(os_type);
```

4.4 Artifact Format Abstraction Layer

Different OS types require different artifact formats. The abstraction layer provides:

1. **Artifact Type Registry** — each adapter declares the artifact types it supports (e.g., `zip`, `msi`, `deb`, `swu`, `firmware_bin`)
2. **Format Validators** — per-artifact-type validation logic (ZIP structure, MSI digital signature, DEB control file, SWU description file)
3. **Delta Generators** — per-artifact-type delta generation (bsdiff for raw binaries, `brillo_update_payload` for Android OTA, `rpm-ostree static-delta` for OSTree)
4. **Storage Layout** — artifacts are stored with OS-type-aware directory structure: `artifacts/{os_type}/{artifact_id}/{filename}`

```
// ArtifactFormatValidator validates an artifact's internal
        structure.
type ArtifactFormatValidator interface {
    // Validate checks the artifact file at the given path.
    Validate(ctx context.Context, path string, artifact
        ArtifactInfo) (*ValidationResult, error)
    // SupportedTypes returns the artifact types this
        validator handles.
    SupportedTypes() []string
}
```

---

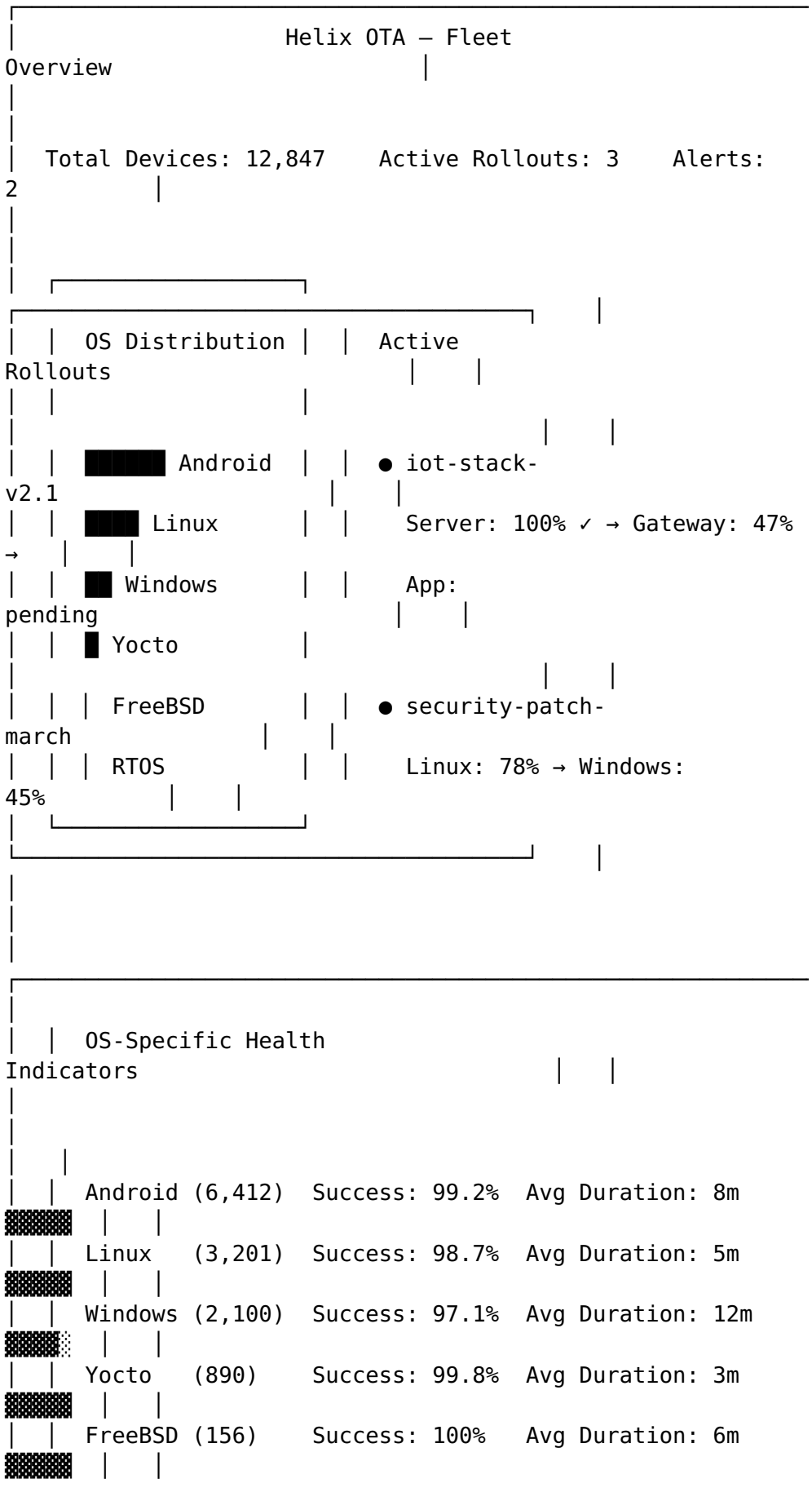
5. Cross-OS Dashboard

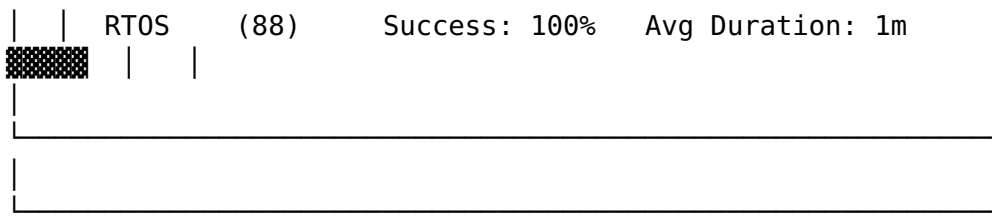
5.1 Unified Fleet View

The 2.0.0 dashboard presents a unified view of all devices across all OS types in a single page. The fleet overview shows:

- **Total devices** across all OS types
- **Devices per OS type** with color-coded indicators
- **Active rollouts** across all OS types with cross-OS dependency status
- **Global health metrics**: aggregate success rate, failure rate, average update duration

- **OS distribution pie chart** showing the percentage breakdown of the fleet by OS type





5.2 OS-Specific Metrics and Health Indicators

Each OS type has both universal metrics (shared across all OS types) and OS-specific metrics:

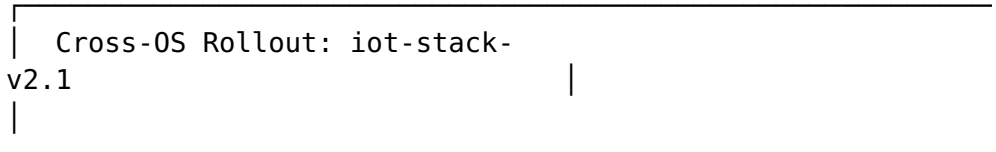
Universal Metrics (all OS types): - Update success rate (percentage of devices that successfully applied an update) - Update failure rate (by error code) - Average update duration (download + verify + install) - Fleet coverage (percentage of devices on the latest version) - Rollout progress (percentage of target devices updated)

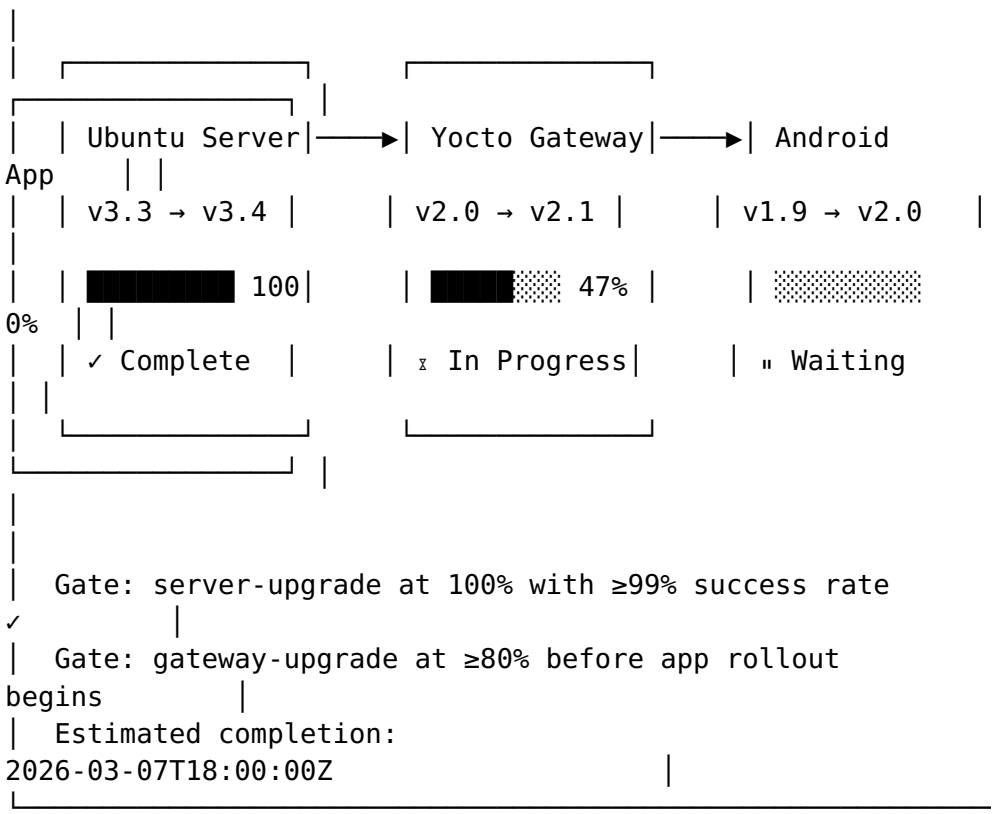
OS-Specific Metrics:

OS Type	Specific Metrics
Android	A/B slot status, `update_engine` error codes, boot reason, partition usage
Linux (APT)	Package conflict count, broken package count, autoremovable count
Linux (rpm-ostree)	Layered package count, base commit hash, deployment count
Windows	Windows Update error codes, MSI return codes, System Restore point status
FreeBSD	Boot environment count, `freebsd-update` status, pkg audit vulnerabilities
Yocto	SWUpdate status, partition layout version, bootloader version
RTOS	MCUboot image status, swap type, image confirm status

5.3 Cross-OS Rollout Coordination Dashboard

The coordination view shows the dependency graph between active rollouts across different OS types:





5.4 OS Distribution Pie Charts and Trends

The dashboard provides time-series visualization of OS distribution trends, showing how the fleet composition changes over time as devices are added, updated, or decommissioned:

- **Pie chart**: Current OS distribution (interactive — click to drill into OS-specific device list)
- **Stacked area chart**: OS distribution over time (last 30 days, 90 days, 1 year)
- **Sankey diagram**: Version flow showing how devices move between versions across OS types during a coordinated rollout

---

6. New Submodules for 2.0.0

6.1 helix-plugin-sdk

Property	Value
Repository (GitHub)	`HelixDevelopment/helix-plugin-sdk`
Go module	`dev.helix.ota.pluginsdk`
Language	Go 1.22+

Property	Value
License	Apache 2.0

The Plugin SDK is the public-facing development kit for building OS adapter plugins. It contains:

1. **`pluginsdk` package** — The `OSAdapter` interface, all type definitions (`AdapterMetadata`, `DeviceInfo`, `UpdateInfo`, `PreparedArtifact`, `Progress`, `VerificationResult`), and sentinel errors
2. **`conformance` package** — The `ConformanceSuite` test harness that validates adapter implementations
3. **`mock` package** — Mock implementations of all SDK interfaces for adapter testing
4. **`helper` package** — Utility functions for common adapter operations (hash computation, progress reporting, retry logic)
5. **`helix-adapter-init` CLI** — Scaffolding tool that generates a new adapter project
6. **`helix-adapter-build` CLI** — Build tool that compiles adapters to native `.so` or WASM `.wasm`
7. **`helix-adapter-test` CLI** — Local testing tool that runs adapters against a mock server

Package Structure:

```
helix-plugin-sdk/
├── pkg/
│   ├── pluginsdk/
│   │   ├── adapter.go          # OSAdapter interface + all
types
│   │   ├── errors.go           # Sentinel errors
│   │   └── doc.go
│   ├── conformance/
│   │   ├── suite.go            # ConformanceSuite
implementation
│   │   ├── suite_test.go
│   │   └── testcases/
│   │       ├── metadata_test.go
│   │       ├── initialize_test.go
│   │       ├── check_test.go
│   │       ├── prepare_test.go
│   │       ├── apply_test.go
│   │       ├── verify_test.go
│   │       ├── rollback_test.go
│   │       ├── health_test.go
│   │       └── shutdown_test.go
│   ├── mock/
│   │   ├── mock_adapter.go     # Generated mock for
OSAdapter
│   │   ├── mock_server.go      # Mock Helix OTA server
```

```
│   │   └── mock_device.go       # Mock device with
configurable behavior
│   └── helper/
│       ├── hash.go              # Streaming hash helpers
│       ├── progress.go          # Progress channel helpers
│       ├── retry.go             # Retry with backoff
│       └── config.go            # Config parsing helpers
├── cmd/
│   ├── helix-adapter-init/
│   │   └── main.go
│   ├── helix-adapter-build/
│   │   └── main.go
│   └── helix-adapter-test/
│       └── main.go
├── examples/
│   └── adapter-freebsd/         # Working example adapter
│       ├── adapter.go
│       ├── plugin.yaml
│       └── README.md
├── go.mod
├── Makefile
├── README.md
├── CLAUDE.md
└── CONTRIBUTING.md
```

6.2 helix-plugin-registry

Property	Value
Repository (GitHub)	`HelixDevelopment/helix-plugin-registry`
Go module	`dev.helix.ota.registry`
Language	Go 1.22+
License	Apache 2.0

The Plugin Registry provides storage, versioning, and distribution of adapter plugins. It supports two deployment modes:

1. **Local mode** — Plugins stored on the server's filesystem; suitable for air-gapped deployments
2. **Remote mode** — Plugins stored in S3-compatible storage; served via the registry API; supports signed plugin distribution

Key Features: - Plugin versioning with semantic version constraints - Plugin signature verification (RSA-4096) - Plugin download with integrity check (SHA-256) - Plugin

search by OS type, capability, or keyword - Plugin deprecation and migration guides - Automatic update notifications for installed plugins

6.3 helix-adapter-freebsd

Property	Value
Repository (GitHub)	`HelixDevelopment/helix-adapter-freebsd`
Go module	`dev.helix.ota.adapter.freebsd`
Language	Go 1.22+
License	Apache 2.0

FreeBSD adapter implementing `OSAdapter`. Key design decisions:

- **Update mechanism**: `freebsd-update` for base system, `pkg upgrade` for ports/packages
- **Artifact format**: `TXZ` (for base system updates), `PKG` (for package updates)
- **Rollback**: FreeBSD Boot Environments (BE) via `beadm` — each update creates a new BE; rollback is `beadm activate <old-be>`
- **Verification**: SHA-256 hash of TXZ verified against `freebsd-update` metadata; RSA signature on `freebsd-update` manifest
- **Post-install verification**: Check that the new BE boots successfully by monitoring `devd` events for `kernel` boot notifications

```
// FreeBSDAdapter implements OSAdapter for FreeBSD systems.
type FreeBSDAdapter struct {
    config     FreeBSDConfig
    httpClient *http.Client
    logger     *zap.Logger
}

func (a *FreeBSDAdapter) Metadata()
        pluginsdk.AdapterMetadata {
    return pluginsdk.AdapterMetadata{
        Name:          "helix-adapter-freebsd",
        Version:       "1.0.0",
        OSType:        "freebsd",
        OSDisplayName: "FreeBSD",
        Capabilities:  []string{"full_update", "rollback",
        "package_update"},
        APIVersion:    "2.0.0",
    }
}
```

```
func (a *FreeBSDAdapter) ApplyUpdate(
    ctx context.Context,
    device pluginsdk.DeviceInfo,
    artifact pluginsdk.PreparedArtifact,
    progressChan chan<- pluginsdk.Progress,
) error {
    // 1. Create a new boot environment (BE) as a snapshot
    beName := fmt.Sprintf("helix-ota-%s",
        artifact.ArtifactID[:8])
    if err := a.createBootEnvironment(ctx, beName); err !=
        nil {
        return fmt.Errorf("create BE: %w", err)
    }

    progressChan <- pluginsdk.Progress{Stage: "installing",
        ProgressPercent: 25, Message: "Boot environment
        created"}

    // 2. Install update into the new BE
    if err := a.installIntoBE(ctx, beName, artifact); err !
        = nil {
        // Attempt to destroy the failed BE
        _ = a.destroyBootEnvironment(ctx, beName)
        return fmt.Errorf("install into BE: %w", err)
    }

    progressChan <- pluginsdk.Progress{Stage: "installing",
        ProgressPercent: 75, Message: "Update installed
        into BE"}

    // 3. Activate the new BE for next boot
    if err := a.activateBootEnvironment(ctx, beName); err !
        = nil {
        return fmt.Errorf("activate BE: %w", err)
    }

    progressChan <- pluginsdk.Progress{Stage: "finalizing",
        ProgressPercent: 100, Message: "Boot environment
        activated"}
    return nil
}

func (a *FreeBSDAdapter) Rollback(ctx context.Context,
        device pluginsdk.DeviceInfo) error {
    // Activate the previous boot environment
    return a.activateBootEnvironment(ctx,
        a.getPreviousBE(ctx))
}
```

6.4 helix-adapter-macos

Property	Value
Repository (GitHub)	`HelixDevelopment/helix-adapter-macos`
Go module	`dev.helix.ota.adapter.macos`
Language	Go 1.22+
License	Apache 2.0

macOS adapter (targeting 2.1.0 release, scaffolded in 2.0.0). Key design decisions:

- **Update mechanism**: `softwareupdate --install` for macOS system updates; `installer -pkg` for custom packages
- **Artifact format**: PKG (macOS installer package), DMG (disk image for app distribution)
- **Rollback**: APFS snapshot-based rollback — macOS creates an APFS snapshot before each system update; `diskutil apfs restoreSnapshot` reverts to the pre-update state
- **Verification**: Apple code signing verification (`codesign --verify`), notary ticket validation (`spctl --assess`)
- **Restrictions**: macOS System Integrity Protection (SIP) may limit what the adapter can modify; the adapter runs as a LaunchDaemon with root privileges

6.5 helix-adapter-yocto

Property	Value
Repository (GitHub)	`HelixDevelopment/helix-adapter-yocto`
Go module	`dev.helix.ota.adapter.yocto`
Language	Go 1.22+ (client), C (SWUpdate IPC integration)
License	Apache 2.0

Yocto/OpenEmbedded adapter for embedded Linux devices. Key design decisions:

- **Update mechanism**: SWUpdate (primary), Mender client (alternative), raw `dd` to partition (fallback)
- **Artifact format**: SWU (SWUpdate container: `sw-description` + images + scripts)
- **Rollback**: Dual-bank A/B partition layout; bootloader (U-Boot) environment variable controls active bank; fallback on boot failure

- **Verification**: SHA-256 hash of each image in `sw-description`, RSA signature on `sw-description` itself
- **Delta support**: bsdiff between partition images; SWUpdate supports incremental updates via `chained handler`
- **SWUpdate IPC**: The adapter communicates with SWUpdate via its local socket interface (`/tmp/swupdateprog`) for progress reporting and control

```
// YoctoAdapter implements OSAdapter for Yocto/OpenEmbedded
        devices.
type YoctoAdapter struct {
    config   YoctoConfig
    swuPath  string // path to swupdate binary
    logger   *zap.Logger
}

func (a *YoctoAdapter) Metadata() pluginsdk.AdapterMetadata
        {
    return pluginsdk.AdapterMetadata{
        Name:          "helix-adapter-yocto",
        Version:       "1.0.0",
        OSType:        "yocto",
        OSDisplayName: "Yocto/OpenEmbedded",
        Capabilities:  []string{"full_update",
        "delta_update", "rollback", "atomic_install"},
        APIVersion:    "2.0.0",
    }
}

func (a *YoctoAdapter) ApplyUpdate(
    ctx context.Context,
    device pluginsdk.DeviceInfo,
    artifact pluginsdk.PreparedArtifact,
    progressChan chan<- pluginsdk.Progress,
) error {
    // 1. Stream artifact to SWUpdate via IPC
    swuClient, err := swupdate.NewIPCClient("/tmp/
        swupdateprog")
    if err != nil {
        return fmt.Errorf("connect to swupdate: %w", err)
    }

    // 2. Send artifact to SWUpdate for installation
    if err := swuClient.Install(ctx, artifact.DownloadURL,
        func(progress float64) {
        progressChan <- pluginsdk.Progress{
            Stage:           "installing",
            ProgressPercent: progress,
            Message:         "SWUpdate applying image",
        }
    }); err != nil {
        return fmt.Errorf("swupdate install: %w", err)
```

```
    }

    // 3. Update U-Boot environment to boot from new
        partition
    if err := a.setActivePartition(ctx); err != nil {
        return fmt.Errorf("set active partition: %w", err)
    }

    return nil
}
```

6.6 helix-adapter-rtos

Property	Value
Repository (GitHub)	`HelixDevelopment/helix-adapter-rtos`
Go module	`dev.helix.ota.adapter.rtos`
Language	Go 1.22+ (server-side adapter logic), C (MCUboot integration via CGo)
License	Apache 2.0

RTOS adapter for real-time operating systems (FreeRTOS, Zephyr). Key design decisions:

- **Update mechanism**: MCUboot for both FreeRTOS and Zephyr; custom bootloader support via adapter extension points
- **Artifact format**: Raw firmware binary (`.bin`) or Intel HEX (`.hex`); padded to flash sector size
- **Rollback**: MCUboot's built-in image revert mechanism — if the new image fails to confirm within a watchdog timeout, MCUboot automatically reverts to the previous image
- **Verification**: SHA-256 hash of firmware binary verified by MCUboot's TLV area; RSA-3072 or ECDSA-P256 signature in MCUboot image header
- **Constraints**: RTOS devices typically have very limited resources (256KB RAM, 1MB flash). The adapter must handle: chunked firmware delivery, low-bandwidth connections (LoRa, BLE), power-loss safety during flash write
- **Transport**: The RTOS adapter uses a lightweight HTTP/CoAP client on the device side; the server-side adapter generates the appropriately formatted firmware image with MCUboot headers

```go
// RTOSAdapter implements OSAdapter for RTOS devices
        (FreeRTOS, Zephyr).
type RTOSAdapter struct {
    config   RTOSConfig
    logger   *zap.Logger
}

func (a *RTOSAdapter) Metadata() pluginsdk.AdapterMetadata {
    return pluginsdk.AdapterMetadata{
        Name:          "helix-adapter-rtos",
        Version:       "1.0.0",
        OSType:        "rtos",
        OSDisplayName: "RTOS (FreeRTOS/Zephyr)",
        Capabilities:  []string{"full_update",
        "delta_update", "rollback", "low_bandwidth"},
        APIVersion:    "2.0.0",
    }
}

func (a *RTOSAdapter) PrepareArtifact(
    ctx context.Context,
    artifact pluginsdk.ArtifactInfo,
    device pluginsdk.DeviceInfo,
) (*pluginsdk.PreparedArtifact, error) {
    // Wrap firmware binary with MCUboot header
    // MCUboot header format: magic, load_addr, hdr_size,
        img_size, flags, version
    mcuImage, err := mcuboot.WrapImage(artifact,
        device.Metadata)
    if err != nil {
        return nil, fmt.Errorf("mcuboot wrap: %w", err)
    }

    // Sign the MCUboot image
    signedImage, err := mcuboot.SignImage(mcuImage,
        a.config.SigningKey)
    if err != nil {
        return nil, fmt.Errorf("mcuboot sign: %w", err)
    }

    // Upload signed image to artifact storage
    downloadURL, err := a.uploadSignedImage(ctx,
        signedImage)
    if err != nil {
        return nil, fmt.Errorf("upload signed image: %w",
        err)
    }

    return &pluginsdk.PreparedArtifact{
        ArtifactID:  artifact.ArtifactID,
        DownloadURL: downloadURL,
        SHA256:      signedImage.SHA256,
        SizeBytes:   signedImage.Size,
```

```
        InstallParams: map[string]interface{}{
            "mcuboot_slot":
        device.Metadata["active_slot"], // "slot_0" or
        "slot_1"
            "chunk_size":      4096, // Match flash sector
        size
            "confirm_timeout": 300,  // seconds to confirm
        image or MCUboot reverts
        },
    }, nil
}
```

---

7. Roadmap Beyond 2.0.0

7.1 Version Timeline

```
2.0.0 ——— 2.1.0 ——— 2.2.0 ——— 2.3.0 ——— 3.0.0
  |          |          |          |          |
  |          |          |          |          |
  Universal  macOS      Embedded/  iOS        AI-Powered
  Plugin     Adapter    RTOS       (limited)  Scheduling
  Arch.      Stable     Production Adapter    & Predictive
  + FreeBSD             + MCUboot             Rollback
  + Yocto               + OTA mgmt
  + RTOS
  scaffold              for tiny
                        devices
```

7.2 Version 2.1.0 — macOS Support

Estimated Complexity: Large (8–10 weeks)

Feature	Description	Acceptance Criteria
macOS System Updates	Trigger macOS system updates via `softwareupdate`	Client runs `softwareupdate --install --all`; supports selective update; reports available and installed updates
PKG Distribution	Install custom PKG packages via `installer`	Client downloads and installs PKG; verifies Apple code signature; reports success/ failure
APFS Snapshot Rollback	Rollback via APFS snapshot restore	Client creates APFS snapshot before update; restores

Feature	Description	Acceptance Criteria
		snapshot on failure; snapshot creation < 5 seconds
LaunchDaemon Client	Client runs as macOS LaunchDaemon	Installed in `/Library/LaunchDaemons/`; auto-starts on boot; runs as root; supports `launchctl load/unload`
Notarization Integration	Validate notarized apps before install	Client checks notary ticket via `spctl --assess`; rejects non-notarized apps unless overridden by admin policy

New Submodules: `helix-adapter-macos` (promoted from scaffold to production)

Risk: Apple's System Integrity Protection may restrict the adapter's ability to modify system files; sandboxing restrictions on macOS may prevent certain operations. Mitigation: run as a privileged LaunchDaemon; document all required SIP exceptions.

7.3 Version 2.2.0 — Embedded/RTOS Production

Estimated Complexity: Extra Large (10–14 weeks)

Feature	Description	Acceptance Criteria
MCUboot Production Integration	Full MCUboot integration for FreeRTOS and Zephyr	Adapter generates signed MCUboot images; device-side client confirms image after boot; MCUboot revert on failure
CoAP Transport	CoAP-based update delivery for constrained networks	Supports block-wise transfer (RFC 7959); works over LoRa and BLE; resume after disconnection
Low-Bandwidth Delta	Size-optimized delta updates for firmware	Uses bsdiff with custom configuration for small firmware (< 1MB); delta size ≤ 30% of full image for minor changes

Feature	Description	Acceptance Criteria
Firmware Quality Verification	Post-flash verification beyond MCUboot	CRC32 of each flash sector verified after write; boot-time self-test passes before image confirmation
Fleet Orchestration for Constrained Devices	Staged rollout optimized for power-constrained devices	Respects device sleep schedules; batches updates during active windows; handles devices with intermittent connectivity

New Submodules: None (extending `helix-adapter-rtos`)

Risk: RTOS device heterogeneity is extreme — every device has a different flash layout, bootloader, and communication stack. Mitigation: device profiles define all hardware-specific parameters; adapter generates firmware images per-profile.

7.4 Version 2.3.0 — iOS Support (Limited by Apple Restrictions)

Estimated Complexity: Medium (4–6 weeks)

Feature	Description	Acceptance Criteria
MDM Integration	Manage iOS devices via Apple MDM protocol	Server sends MDM commands for OS update policy (defer, enforce, schedule); receives compliance reports; supports supervised and unsupervised devices
App-Level Updates	Distribute custom enterprise apps via OTA	Server manages app catalog; devices install/update apps via MDM; supports in-house and App Store apps
OS Update Policy	Enforce OS version compliance via MDM	Server sets target OS version; MDM enforces deadline; devices report current OS version; non-compliant devices flagged
Compliance Dashboard		Dashboard shows OS version distribution,

Feature	Description	Acceptance Criteria
	iOS-specific compliance reporting	compliance rate, policy violation count, deferred update count

New Submodules: `helix-adapter-ios`

Critical Limitation: Apple does not allow third-party systems to deliver iOS OS updates. The iOS adapter can only manage update policies (when to install Apple's updates) and app-level updates. It cannot replace Apple's OTA mechanism. This is an Apple-imposed restriction, not a technical limitation.

7.5 Version 3.0.0 — AI-Powered Update Scheduling and Predictive Rollback

Estimated Complexity: Extra Large (14–20 weeks)

Feature	Description	Acceptance Criteria
AI Scheduling Engine	ML model predicts optimal update windows based on device usage patterns	Model trained on historical check-in data; predicts per-device optimal update time with ≥ 80% accuracy; reduces failed updates due to user activity by ≥ 40%
Predictive Rollback	ML model detects early signs of update failure and triggers automatic rollback before user impact	Model analyzes post-update telemetry in real-time; detects anomalies within 30 seconds of boot; triggers rollback before user notices degradation; false positive rate < 2%
Anomaly Detection v2	Replace rule-based anomaly detection with ML-based anomaly detection	Unsupervised anomaly detection on device telemetry streams; detects novel failure modes not covered by rules; reduces mean time to detection by ≥ 60%
Smart Rollout Pacing	ML model adjusts rollout speed based on real-time fleet health signals	Model monitors success rate, device health, and error patterns; automatically slows or pauses rollouts when risk increases; achieves same

Feature	Description	Acceptance Criteria
		rollout completion time with ≤ 50% of the failures
Natural Language Rollout	Create rollouts via natural language descriptions	Admin describes rollout intent in plain English; LLM generates rollout configuration; admin reviews and confirms; supports multi-OS coordination descriptions

New Submodules: - `helix-ai-engine` — ML model training, serving, and inference pipeline - `helix-ai-dashboard` — AI-specific dashboard views for model monitoring and confidence scoring

Risk: ML models require significant training data; cold-start problem for new device types. Mitigation: rule-based fallback for device types with insufficient data; synthetic data generation for bootstrapping; transfer learning from similar OS types.

Appendix A — Go Interface Reference

Complete Interface Hierarchy

```
OSAdapter (primary plugin interface)
├── Metadata() AdapterMetadata
├── Initialize(ctx, config) error
├── CheckForUpdate(ctx, DeviceInfo) (*UpdateInfo, error)
├── PrepareArtifact(ctx, ArtifactInfo, DeviceInfo)
(*PreparedArtifact, error)
├── ApplyUpdate(ctx, DeviceInfo, PreparedArtifact, chan
Progress) error
├── VerifyUpdate(ctx, DeviceInfo) (*VerificationResult,
error)
├── Rollback(ctx, DeviceInfo) error
├── HealthCheck(ctx) error
└── Shutdown(ctx) error

AdapterRegistry (plugin storage interface)
├── ListAvailable(ctx) ([]RegistryEntry, error)
├── Download(ctx, name, version) (string, error)
├── Upload(ctx, entry, binaryPath, manifestPath) error
├── GetVersions(ctx, name) ([]string, error)
└── VerifyIntegrity(ctx, name, version) error
```

```
AdapterManager (lifecycle management)
├── LoadAndRegister(ctx, pluginPath) error
├── GetAdapter(osType) (OSAdapter, error)
├── UnloadAdapter(ctx, osType) error
├── ListAdapters() []AdapterStatus
└── HealthMonitor(ctx) error

ArtifactFormatValidator (artifact validation)
├── Validate(ctx, path, ArtifactInfo) (*ValidationResult,
error)
└── SupportedTypes() []string

UniversalUpdatePipeline (update execution)
└── Execute(ctx, DeviceInfo) error
```

Appendix B — Plugin Manifest Specification

```yaml
# plugin.yaml — Required file in every adapter plugin
        package
# Schema version: 2.0.0

apiVersion: "2.0.0"           # Must match server's
        supported plugin API version
name: "helix-adapter-freebsd" # Unique adapter name (kebab-
        case)
version: "1.0.0"              # Semantic version
runtime: "wasm"               # "native" or "wasm"
os_type: "freebsd"            # OS type identifier (unique
        key for adapter registration)
os_display_name: "FreeBSD"    # Human-readable OS name
description: "FreeBSD OTA adapter with boot environment
        support"
author: "HelixDevelopment"
license: "Apache-2.0"
homepage: "https://github.com/HelixDevelopment/helix-
        adapter-freebsd"
min_server_version: "2.0.0"  # Minimum Helix OTA server
        version

capabilities:
  - full_update               # Full system update support
  - package_update            # Package-level update support
  - rollback                  # Rollback support
  # - delta_update            # Delta/differential update
        support
  # - low_bandwidth           # Optimized for constrained
        networks
  # - atomic_install          # Atomic (all-or-nothing)
        installation
```

```yaml
artifact_types:               # Artifact formats this
        adapter accepts
  - "txz"                     # FreeBSD base system update
  - "pkg"                     # FreeBSD package update

permissions:                  # Permissions requested by the
        adapter
  - "filesystem:/var/db/freebsd-update"  # Filesystem access
  - "process:freebsd-update"             # Process execution
  - "process:beadm"                      # Boot environment
        management
  - "process:pkg"                        # Package
        management
  - "network:egress"                     # Outbound network
        access (for downloads)

config_schema:                # JSON Schema for adapter-
        specific configuration
  type: object
  properties:
    update_type:
      type: string
      enum: ["base", "packages", "both"]
      default: "both"
    be_prefix:
      type: string
      default: "helix-ota"
    max_be_count:
      type: integer
      default: 5
      description: "Maximum number of boot environments to
        retain"
  required: ["update_type"]

signature:                    # Plugin signature (RSA-4096
        over binary + manifest)
  algorithm: "RSA-SHA256"
  public_key_id: "helix-ca-2026"
  value: "BASE64_ENCODED_SIGNATURE..."

entry_point:                  # Entry point for WASM or
        native plugin
  wasm: "adapter.wasm"
  native: "adapter.so"
```

Appendix C — Risk Assessment Matrix

#	Risk	Probability	Impact	Mitigation	Owner
R1		**High**	**Critical**		Platform

#	Risk	Probability	Impact	Mitigation	Owner
	Go `plugin.Open` compatibility limitations (same Go version, same build flags)			WASM as primary plugin runtime; native mode only for first-party adapters built in same CI pipeline	
R2	Malicious plugin compromises server	**Medium**	**Critical**	WASM sandboxing (wazero capability-based security); plugin signature verification; resource limits (CPU, memory, file descriptors)	Security
R3	Breaking change to OSAdapter API alienates third-party developers	**Medium**	**High**	API versioning in manifest; backward-compatible wrappers; 2-version deprecation period	Platform
R4	Cross-OS dependency graph becomes intractable	**Medium**	**Medium**	Limit dependency depth to 3 levels; cycle detection at rollout creation; dependency visualization tool	Platform
R5	Plugin load-time performance regression	**Low**	**Medium**	Lazy-load adapters on first device check-in for that OS type; cache compiled WASM instances; benchmark ≤	Platform

#	Risk	Probability	Impact	Mitigation	Owner
				100ms load, ≤ 1ms per call	
R6	FreeBSD adapter incompatibility across FreeBSD versions (13.x, 14.x, 15.x)	**Medium**	**Medium**	Version-specific adapter configurations; test matrix covers 13-RELEASE, 14-RELEASE, 15-CURRENT	FreeBSD
R7	Yocto SWUpdate IPC protocol changes between SWUpdate versions	**Medium**	**High**	Pin SWUpdate version in device profile; adapter queries SWUpdate version on init; version-specific IPC handling	Yocto
R8	RTOS MCUboot image format incompatibility across MCUboot versions	**Low**	**High**	MCUboot image format is stable (v1.x); adapter validates MCUboot version before generating images; fallback to raw flash	RTOS
R9	Hot-swap adapter causes in-flight update failures	**Medium**	**High**	Drain mode waits for all in-flight updates to complete; 30-second timeout with forced shutdown; device retries on next check-in	Platform
R10	Third-party adapter registry becomes	**Medium**	**Critical**	All registry plugins must be signed by a trusted CA; registry verifies	Security

#	Risk	Probability	Impact	Mitigation	Owner
	supply-chain attack vector			signature on download; admin must explicitly approve new CAs	

Appendix D — Testing Strategy

D.1 Test Categories

Category	Target	Tool	Notes
Unit Tests	85% line coverage	Go `testing`	Every adapter method, pipeline stage, lifecycle transition
Mutation Tests	85% mutation score	`go-mutesting`	Mandatory gate per HelixConstitution §1.1
Adapter Conformance	All conformance test cases pass	`pluginsdk.ConformanceSuite`	Every adapter (first-party and third-party) must pass
Integration Tests	Full adapter lifecycle against mock server	`testcontainers-go`	Register → configure → activate → execute → shutdown
E2E Tests	Full update cycle on real OS	Per-OS test harness	FreeBSD VM, Yocto QEMU, RTOS QEMU, macOS VM
Plugin Security Tests	Sandbox escape, privilege escalation	Custom pen-test suite	WASM sandbox, file access, network access, process execution
Cross-OS Rollout Tests	Coordinated rollout across 3+ OS types	Custom multi-device simulator	Verify dependency ordering, gate enforcement, rollback cascading

Category	Target	Tool	Notes
Performance Tests	Plugin load ≤ 100ms, call overhead ≤ 1ms	`go test -bench`	Benchmark every adapter method
Chaos Tests	Server restart mid-update, network partition	Custom harness	Device must resume or safely rollback on all OS types

D.2 Adapter Conformance Test Matrix

Test Case	Validates	Mu Pas
`TestMetadataReturnsValidStruct`	Adapter returns complete, valid metadata	All ada
`TestInitializeAcceptsConfig`	Adapter initializes with server config	All ada
`TestInitializeRejectsInvalidConfig`	Adapter validates config and rejects invalid	All ada
`TestCheckForUpdateReturnsUpdateInfo`	Adapter returns update info when available	All ada
`TestCheckForUpdateReturnsNilWhenNoUpdate`	Adapter returns nil when no update	All ada
`TestPrepareArtifactValidatesCompatibility`	Adapter validates artifact-device compatibility	All ada
`TestPrepareArtifactRejectsIncompatibleArtifact`	Adapter rejects incompatible artifacts	All ada
`TestApplyUpdateReportsProgress`	Adapter sends progress updates	Ada with full
`TestApplyUpdateReturnsErrorOnFailure`	Adapter returns error on install failure	Ada with full
`TestVerifyUpdateConfirmsSuccess`	Adapter confirms successful update	Ada with full
`TestVerifyUpdateDetectsFailure`	Adapter detects failed update	Ada with full

Test Case	Validates	Mu Pas
`TestRollbackRevertsToPreviousState`	Adapter reverts to previous state	Ada with `roll`
`TestRollbackReturnsErrNotSupportedIfUnsupported`	Adapter returns `ErrRollbackNotSupported` if unsupported	Ada with `roll`
`TestHealthCheckReturnsNilWhenHealthy`	Adapter reports healthy	All ada
`TestHealthCheckReturnsErrorWhenUnhealthy`	Adapter reports unhealthy	All ada
`TestShutdownCleansUpResources`	Adapter cleans up on shutdown	All ada
`TestConcurrentCallsAreSafe`	Adapter handles concurrent method calls safely	All ada
`TestContextCancellationIsRespected`	Adapter respects context cancellation	All ada

> **Document End** — Helix OTA 2.0.0 Multi-OS Universal Design Document
>
> This document is a living specification. Changes require review by two engineers and must pass the anti-bluff verification per HelixConstitution §2.3.